



RISC-V S-level Physical Memory Protection (SPMP)

Editor - Dong Du, Bicheng Yang, RISC-V SPMP Task Group

Version 0.9.1, 6/2026: This document is in the Frozen state. Changes are highly unlikely. A high threshold will be applied, and modifications will only be made in response to critical issues. Any other proposed changes should be addressed through a follow-on extension.

Table of Contents

Preamble	1
Copyright and license information.....	2
Contributors	3
1. Introduction.....	4
2. "Ssmp" Extension for S-level Physical Memory Protection (SPMP)	5
2.1. Extension Dependencies.....	5
2.2. S-level Physical Memory Protection CSRs	5
2.3. Address Matching	7
2.4. Encoding of Permissions	7
2.5. Matching Logic.....	8
2.6. SPMP and Paged Virtual Memory	9
2.7. The Access Method for SPMP CSRs in S-mode	9
2.8. Exceptions	10
3. "Ssmpen" Extension for Optimizing Context Switching of SPMP Entries	12
4. "Smpmpdeleg" Extension for Sharing Hardware Resources between PMP and SPMP	13
4.1. Resource Sharing between PMP and SPMP	13
4.2. The Access Methods for SPMP CSRs in M-mode	14
5. Recommended Programming Guidelines	16
5.1. Static Configuration	16
5.2. Dynamic Reconfiguration	16
5.3. Entry Configuration Recommendations.....	17
5.4. Reconfiguration Non-preemption and Synchronization.....	18

Preamble



This document is in the [Frozen state](#)

Changes are highly unlikely. A high threshold will be applied, and modifications will only be made in response to critical issues. Any other proposed changes should be addressed through a follow-on extension.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2026 by RISC-V International.

Contributors

The proposed SPMP specifications has been contributed to directly or indirectly by:

- Dong Du, Editor <dd_nirvana@sjtu.edu.cn>
- Bicheng Yang, Editor <bichengyang@sjtu.edu.cn>
- José Martins
- Thomas Roecker
- Sandro Pinto
- Manuel Rodriguez
- Luís Cunha
- Rongchuan Liu
- Nick Kossifidis
- Andy Dellow
- Manuel Offenberger
- Allen Baum
- Bill Huffman
- Xu Lu
- Wenhao Li
- Yubin Xia
- Joe Xie
- Paul Ku
- Jonathan Behrens
- Robin Zheng
- Zeyu Mi
- Fabrice Marinet
- Noureddine Ait Said
- Xi Zhao
- Andrew Waterman
- Earl Killian
- Greg Favor
- John Hauser
- Krste Asanović
- Vedvyas Shanbhogue

Chapter 1. Introduction

This document introduces the RISC-V S-level Physical Memory Protection (SPMP) extension. SPMP provides a mechanism for enforcing memory isolation between Supervisor mode (S-mode) and User mode (U-mode) software in systems that implement S-mode but do not use paged virtual memory and thus do not require a Memory Management Unit (MMU).

The RISC-V Privileged Architecture provides Physical Memory Protection (PMP) to enable Machine mode (M-mode) software to control memory accesses by less-privileged modes. However, in systems operating with `satp.MODE=Bare`, there is no standard mechanism that allows S-mode software to enforce memory isolation on U-mode software. As a result, an operating system executing in S-mode cannot independently restrict the physical memory regions accessible to user applications.

In such systems, software is often structured with platform firmware executing in M-mode and an operating system executing in S-mode. This arrangement reduces the amount of software executing with machine privilege while preserving a clear separation between platform and operating-system responsibilities.

SPMP extends the PMP programming model to S-mode. It enables supervisor software to define physical memory regions and associated access permissions for less-privileged software, thus providing memory isolation and fault containment for U-mode applications in systems where paging is not used.

Chapter 2. "Ssmp" Extension for S-level Physical Memory Protection (SPMP)

The RISC-V SPMP mechanism provides per-hart control registers in supervisor mode. These registers define physical memory access privileges—read, write, and execute—for discrete physical memory regions.

A memory access is successful only when permission checks for both S-mode execution environment and SPMP pass. SPMP checks can be performed by the hardware in parallel with PMA and PMP checks. SPMP exceptions take priority over PMP or PMA exceptions. Consequently, if a memory access violates both SPMP and PMP/PMA rules, only the SPMP exception is reported.

SPMP checks are enforced on all memory accesses with effective privilege modes less privileged than M-mode. SPMP can be configured to grant permissions to U-mode, which has none by default, and to revoke permissions from S-mode.

If the Shbare extension is implemented in conjunction with Ssmp, when $V=1$ and $hgap.MODE=Bare$, SPMP enforces access checks on all memory accesses from VS and VU-modes, effectively applying SPMP protections to guest execution contexts. Additionally, the Hypervisor Virtual Machine Load and Store instructions (HLV, HLVX, HSV), when executed from the HS-mode, or U-mode (when $hstatus.HU=1$), are also subject to SPMP checks when $V=1$ and $hgap.MODE=Bare$.

2.1. Extension Dependencies

1. The `Sscsrind` extension for indirect CSR access must be implemented.
2. The `sstatus.SUM` (permit Supervisor User Memory access) bit must be **writable**.
3. The `sstatus.MXR` (Make eXecutable Readable) bit must be **writable**.



The Ssmp extension does not provide ISA support to accelerate virtualization of the SPMP functionality.

2.2. S-level Physical Memory Protection CSRs

Each SPMP entry is composed of an SXLEN-bit configuration register and an associated SXLEN-bit address register. The SPMP registers are accessible only via the indirect access mechanism. For details on CSR numbers and how to access them, see [Section 2.7](#). In TOR mode, an entry also uses the address register of the preceding entry to define its range (see [Section 2.3](#)). Implementations support from 1 to 64 SPMP entries.



An SPMP entry refers to the register pair `smppcfg[i]` and `smppaddr[i]`.

An SPMP rule is defined by the contents of an `smppcfg` register and its corresponding `smppaddr` register(s). These registers collectively define a protected physical memory region and its access constraints.

The SPMP address registers, named `smppaddr0` through `smppaddr63`, share the same layout as the M-mode PMP architecture. On RV32 systems, each `smppaddr` register encodes a 34-bit physical address from bit 33 down to bit 2, as illustrated in [Figure 1](#). On RV64 systems, each `smppaddr` register encodes a 56-bit physical address from bit 55 down to bit 2, as shown in [Figure 2](#). An implementation may support fewer address bits on systems with a smaller physical address space. All writable SPMP entries

should implement the same number of address bits. Since not all physical address bits must be implemented, the SPMP address registers are considered WARL, with exceptions defined by granularity rules. Refer to the "RISC-V Privileged Architecture", Physical Memory Protection, Address Matching.



If `spmpaddr` is narrower than the number of supported physical address bits, some physical addresses may not be representable in an SPMP entry. Since S-mode cannot access any memory unless an SPMP entry permits it, any memory region that cannot be addressed by an SPMP entry is inaccessible to S-mode.

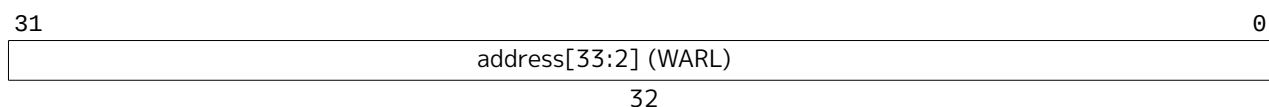


Figure 1. SPMP address register format, RV32.

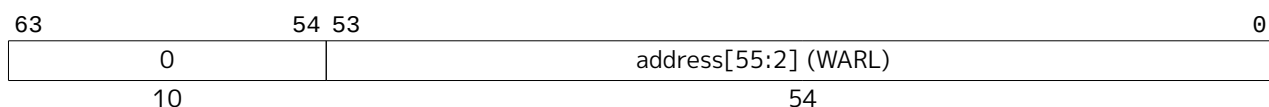


Figure 2. SPMP address register format, RV64.

Every SPMP entry contains an SXLEN-bit configuration register, `spmpcfg[i]`. Figure 3 illustrates the layout of `spmpcfg[i]`. Permission rules and their encodings are detailed in Section 2.4.

1. The R, W, and X bits govern permissions for read, write, and instruction execution, respectively.
2. The A field, which defines the address-matching mode, is detailed in Section 2.3.
3. Bits 5 and 6 are reserved for future standard use.
4. The L (lock) bit designates an entry as locked. Setting the L bit locks the SPMP entry, regardless of the A field's setting (even OFF). Attempts to write to locked `spmpcfg[i]` and `spmpaddr[i]` registers using the `siselect` CSR are ignored. If a locked entry has `spmpcfg[i].A` set to TOR, writes to the preceding `spmpaddr[i-1]` via `siselect` are also ignored.
5. For any rule not designated as a Shared-Region, the U bit determines if it is U-mode (when set) or S-mode-only (when clear), as explained in Section 2.4.
6. The SHARED bit identifies a rule as a Shared-Region rule.

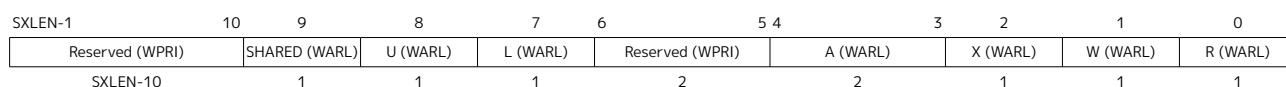


Figure 3. SPMP configuration register format.



M-mode can leverage the L bit to create a sandbox for S-mode software. This is achieved by setting and locking high-priority SPMP entries where `spmpcfg[i].U` is 1. This mechanism effectively thwarts privilege escalation attacks that might try to reconfigure SPMP entries to bypass S-mode restrictions. While PMP/ePMP entries could offer a similar function, the resulting configuration is not identical because PMP does not distinguish between S-mode and U-mode. Moreover, if resource sharing is statically defined (see Section 4.1), there might not be enough PMP/ePMP entries to enforce the intended isolation policy.

2.3. Address Matching

The A field within an SPMP entry's configuration register determines the address-matching mode for its associated `spmpaddr` register. The following table details the A field's encoding.

<code>spmpcfg[i].A</code>	Name	Description
0	OFF	Null region (disabled)
1	TOR	Top of range
2	NA4	Naturally aligned four-byte region
3	NAPOT	Naturally aligned power-of-two region, ≥ 8 bytes

This encoding is consistent with the PMP/ePMP. For comprehensive details, refer to the "Address Matching" subsection for PMP in the "RISC-V Privileged Architecture".

For a rule to be valid, its `spmpcfg[i].A` field must not be OFF. Furthermore, if `spmpcfg[i].A` is set to TOR, the condition `spmpaddr[i-1] < spmpaddr[i]` must hold. Particularly, if `spmpcfg[0].A` is set to TOR, zero is used for the lower bound.

Software can probe the minimum SPMP granularity. This is done by clearing `spmpcfg[i]`, writing all ones to `spmpaddr[i]`, and then reading back the resulting `spmpaddr[i]` value. If G is the index of the least-significant bit set in the result, the granularity is 2^{G+2} bytes.

Software can also determine the implemented physical address bits in `spmpaddr`. This involves setting `spmpcfg[i].A` to 0b11, writing all ones to `spmpaddr[i]`, and reading back the result. (Consult the "NAPOT range encoding in PMP address and configuration registers" table in the "RISC-V Privileged Architecture" for interpretation.)

Because the `spmpcfg[i].A` field is WARL, an implementation is free to hardwire a specific address-matching mode.

2.4. Encoding of Permissions

SPMP supports three distinct rule types: **S-mode-only**, **U-mode** and **Shared-Region**.

1. An **S-mode-only** rule **enforces** accesses for Supervisor mode and **denies** accesses for User mode.
2. A **U-mode** rule is always enforced for User mode accesses. Its behavior for Supervisor mode accesses depends on the `sstatus.SUM` bit.
 - With `sstatus.SUM` set, the rule is enforced for Supervisor mode data accesses, but execution permission is denied (termed **EnforceNoX** in Figure 4). This prevents the OS from executing code residing in memory assigned to an unprivileged process.
 - With `sstatus.SUM` clear, the rule is denied for any Supervisor mode access. This prevents the OS from accessing the memory of an unprivileged process unless a specific code path is followed.
3. The encoding `spmpcfg.SHARED == 1` and `spmpcfg.U == 1` defines a **Shared-Region** rule. For shared regions, the state of the `sstatus.SUM` bit is irrelevant.

4. A **Shared-Region** rule is **enforced** for both Supervisor and User modes, but it imposes the constraint that read and write permissions for User mode are mutually exclusive.
5. The R, W, and X bits collectively form a WARL field.
6. The encodings `spmpcfg.RWX=010`, `spmpcfg.RWX=011`, as well as the combination `spmpcfg.SHARED == 1` and `spmpcfg.U == 0` are all reserved for future standard use.

If the Shbare extension is implemented, when `V=1` and `hgap.MODE=Bare`, the permission encodings remain consistent with those when `spmpcfg.U=1`, but are applied to VS/VU rather than U-mode accesses.

The complete table of encodings and their outcomes is presented in [Figure 4](#):

Shared Rules	spmpcfg.SHARED =0					spmpcfg.SHARED =1		
U/S Rules	spmpcfg.U =1 (U-mode rules)			spmpcfg.U =0 (S-mode rules)		spmpcfg.U =1 (Shared rules)		spmpcfg.U =0 (Shared rules)
Effective Privilege Mode	U mode Access	S mode Access		U mode Access	S mode Access	U mode Access	S mode Access	U/S mode Access
Supervisor User Memory Access	sstatus.SUM is ignored	sstatus.SUM =0	sstatus.SUM =1	sstatus.SUM is ignored		sstatus.SUM is ignored		sstatus.SUM is ignored
RWX=000	Enforce	Deny	EnforceNoX	Deny	Enforce	Enforce	Enforce	Reserved
RWX =100	Enforce	Deny	EnforceNoX	Deny	Enforce	Enforce	Enforce	Reserved
RWX =110	Enforce	Deny	EnforceNoX	Deny	Enforce	Read-only	Enforce	Reserved
RWX =001	Enforce	Deny	EnforceNoX	Deny	Enforce	Enforce	Enforce	Reserved
RWX =101	Enforce	Deny	EnforceNoX	Deny	Enforce	Enforce	Enforce	Reserved
RWX =111	Enforce	Deny	EnforceNoX	Deny	Enforce	Exec-only	Enforce	Reserved
RWX =010	Reserved					Reserved		Reserved
RWX =011	Reserved					Reserved		Reserved

Figure 4. SPMP Encoding Table

Deny: The memory access is blocked and an exception is raised.

Enforce: The R/W/X permissions defined in `spmpcfg` are applied to the access.

EnforceNoX: The R/W permissions from `spmpcfg` are applied, but execute permission is denied.

Reserved: This encoding is reserved for future standard use.

SUM bit: The SPMP mechanism uses the `sstatus.SUM` (permit Supervisor User Memory access) bit to alter the privilege of S-mode load and store operations on physical memory.



The semantics of `sstatus.SUM` within SPMP are consistent with its definition in the Machine-Level ISA (for details, refer to the "Memory Privilege in `mstatus` Register" subsection of the "RISC-V Privileged Architecture").

2.5. Matching Logic

1. SPMP entries are statically prioritized.
2. The lowest-numbered SPMP entry that matches any byte of an access (indicated by an address and the accessed length) determines whether that access is allowed or denied.

3. This matching SPMP entry must match **all** bytes of the access, or the access fails and an instruction, load, or store page-fault exception is generated (see [Section 2.8](#)).
4. This matching is done irrespective of the SHARED, U, R, W, and X bits.
5. If the effective privilege mode of the access is S-mode or U-mode and no SPMP entry matches, but at least one SPMP entry is implemented, the access is denied.
6. Otherwise, each access is checked according to the permission bits in the matching SPMP entry. That access is allowed if it satisfies the permission checking with the encoding corresponding to the access type.

Certain implementations may decompose misaligned memory operations into multiple accesses. In such cases, some of these sub-accesses might succeed before another triggers an exception. Notably, a portion of a misaligned store that passes an SPMP check could become architecturally visible, even if another portion of the same store fails. This behavior may also occur for stores wider than XLEN (e.g., FSD instruction in RV32D), even if the store address is naturally aligned.

SPMP rules are checked for all memory accesses, both implicit and explicit, that originate from S-mode or any less-privileged mode.



The execution environment is expected to configure one or more SPMP entries to grant S-mode its necessary baseline permissions. S-mode software can subsequently constrain these permissions by refining the SPMP entries.

2.6. SPMP and Paged Virtual Memory

SPMP and paged virtual memory are mutually exclusive.

The following table dictates which isolation mechanism is active based on the satp configuration.

satp	Isolation mechanism
satp.mode == Bare with Ssmp	SPMP only
satp.mode == Bare without Ssmp	no S-mode protection checks
satp.mode != Bare	Paged Virtual Memory only

2.7. The Access Method for SPMP CSRs in S-mode

Each value of `siselect` maps to a corresponding set of SPMP CSRs. `sireg` is used to access the `spmpaddr` register, while `sireg2` is used for the `spmpcfg` register. The registers `sireg3` through `sireg6` are reserved (i.e., reads return zero, writes are ignored).

SPMP entries are indexed starting from zero. For instance, in a system with N SPMP entries ($N \leq 64$), `SPMP[0..N-1]` is accessed by writing `0x100+i` to the `siselect` register, where i represents the entry index ($0 \leq i \leq N-1$). An access to an out-of-bounds index via `siselect` will return zero on read and be ignored on write.

siselect value	indirect CSR access of sireg
0x100	sireg → smpaddr[0], sireg2 → smpcfg[0]
0x101	sireg → smpaddr[1], sireg2 → smpcfg[1]
...	...
0x13F	sireg → smpaddr[63], sireg2 → smpcfg[63]



The design choice to map only one SPMP entry per siselect value is motivated by performance. Mapping multiple entries would necessitate a jump table or extra logic to select the correct target sireg register, adding overhead.

For further details on indirect CSR access, refer to the Sscsrind extension specification in the "RISC-V Privileged Architecture".

Indirect accesses to SPMP CSRs are not ordered with respect to each other or with subsequent memory accesses. To enforce ordering for subsequent accesses whose effective privilege mode is S or U, software must execute an SFENCE.VMA instruction with rs1=x0 and rs2=x0, which synchronizes subsequent memory accesses with all preceding SPMP CSR writes. To enforce ordering for subsequent accesses whose effective privilege mode is VS or VU, an HFENCE.GVMA instruction with rs1=x0 and rs2=x0 is required.



Allowing indirect accesses to SPMP CSRs to be not ordered with respect to each other or to subsequent memory accesses enables fast context switching of SPMP registers. While SFENCE.VMA and HFENCE.GVMA instructions normally order preceding stores and subsequent implicit accesses to memory management structures, SPMP entries are also effectively regarded as memory management structures.

2.8. Exceptions

A failed SPMP check triggers an exception whose type corresponds to the memory operation (load, store/AMO, or instruction fetch). Each fault type is assigned a distinct exception code.

The Ssmp extension co-opts the existing page fault exception codes for SPMP violations, as page fault exceptions are commonly delegated to S-mode. S-mode software (e.g., an OS) can differentiate between an SPMP fault and a standard page fault by querying the satp.mode field, given that SPMP and paging are mutually exclusive (see [Section 2.6](#)).

It is important to note that a single instruction can result in multiple memory accesses that are not guaranteed to be atomic relative to each other.

If the Shbare extension is implemented, when a VS/VU access is denied by SPMP, an exception is raised. The type of exception depends on the nature of the access attempt, i.e., whether it is a load, store/AMO, or instruction fetch. SPMP reuses the guest page fault exception codes (20, 21, and 23, for instruction, load, and store/AMO accesses, respectively).

Table of exception codes:

Interrupt	Exception Code	Description
0	12	Instruction page fault
0	13	Load page fault

Interrupt	Exception Code	Description
0	15	Store/AMO page fault
0	20	Instruction guest-page fault
0	21	Load guest-page fault
0	23	Store/AMO guest-page fault



Since, when $V=1$, SPMP is only active when $hgap.MODE=Bare$, SPMP protection and G-stage translation are mutually exclusive. Consequently, when SPMP is active, paged virtual memory translations are disabled, and guest page fault exception codes can be repurposed to indicate guest SPMP access violations.

Chapter 3. "Sspmpen" Extension for Optimizing Context Switching of SPMP Entries

In RV64, a context switch for the SPMP mechanism involves updating up to 64 address registers and 64 configuration registers. The Sspmpen extension, introduced in this chapter, is an optional feature designed to enhance the performance of SPMP context switches.

- For RV64 architectures, it introduces a 64-bit WARL CSR, `spmpen` (CSR number 0x183).
- For RV32 architectures, it introduces an additional 32-bit WARL CSR, `spmpenh` (CSR number 0x193), which serves as an alias for the upper 32 bits of the `spmpen` register.

The activation of each SPMP entry is governed by its corresponding bit in the `spmpen` register. An SPMP entry `i` is considered active only when both the `spmpen[i]` bit is set and the `spmpcfg[i].A` field is non-zero. The `spmpen[i]` bit controls only whether SPMP entry `i` participates in address matching and enforcement; it does not control whether `spmpaddr[i]` or `spmpcfg[i]` is writable.

When an entry `i` is locked, as indicated by `spmpcfg[i].L == 1`, its corresponding `spmpen[i]` bit becomes read-only.

In an implementation featuring 64 PMP entries with N ($N \leq 64$) delegated to the supervisor level, the bits `spmpen[0..N-1]` control their respective SPMP entries `[0..N-1]`. Any attempt to write to the upper bits of the register are ignored.

Writes to the `spmpen` CSR are not ordered with respect to subsequent memory accesses. To enforce ordering, software must execute memory-management fence instructions as described in [Section 2.7](#).

If the Sspmpen extension is present and an entry's addressing mode is set to TOR via `spmpcfg[i].A`, that entry matches any address y that satisfies the following conditions:

1. $\text{spmpaddr}[i-1] \leq y < \text{spmpaddr}[i]$.
2. This address matching is independent of the configuration or activation state of the preceding entry, i.e., `spmpcfg[i-1]` and `spmpen[i-1]`.

The `spmpen` register offers significant benefits for context switch optimization in various scenarios, including the following:



1. *When a hart possesses enough SPMP entries for all its concurrent tasks, memory regions for each task can be statically assigned to a dedicated subset of these entries. Under this configuration, an SPMP context switch is streamlined into a single write operation to `spmpen` (or two writes on RV32 systems: `spmpen` and `spmpenh`). This operation simultaneously deactivates the SPMP entries of the outgoing task while activating those of the incoming task.*
2. *A subset of SPMP entries may be reserved for tasks with strict timing or latency requirements, such as interrupt service routines. This approach guarantees minimal overhead when activating these critical contexts, thereby eliminating the need to dynamically reconfigure SPMP entries during the switch.*

Chapter 4. "Smpmpdeleg" Extension for Sharing Hardware Resources between PMP and SPMP

The similar architecture of PMP and SPMP registers, including their shared address-matching logic, makes hardware reuse a practical approach for resource conservation. This chapter introduces the Smpmpdeleg extension, a mechanism that allows hardware resources to be dynamically allocated between PMP and SPMP.

This extension is mandatory for implementations that support both Ssmp and Sm1p13. The Smcsrind extension for indirect CSR access must be implemented. For the purposes of this specification, Smpmpdeleg assumes an architectural space of 64 PMP entries, although fewer entries may be writable; any non-writable entries behave as read-only zero.

4.1. Resource Sharing between PMP and SPMP

The Smpmpdeleg extension facilitates the delegation of PMP entries for use by S-mode, thereby creating SPMP entries. This delegation is managed by an MXLEN-bit M-mode CSR named mpmmpdeleg (CSR number 0x316), whose layout is detailed in [Figure 5](#).

1. A key component of this CSR is the pmpnum WARL field, which defines the starting index for delegation. All PMP entries with an index equal to or greater than pmpnum are delegated as SPMP entries.
2. If a write to pmpnum specifies a value exceeding the number of writable PMP entries, the field subsequently reads back the total count of writable entries. In such a case, no SPMP entries are delegated.
3. Setting pmpnum to zero delegates all writable PMP entries to SPMP, while setting it to the total number of writable entries delegates none.
4. By default, unless hardwired, pmpnum resets to the total number of writable PMP entries.
5. If no entries are delegated to SPMP, the Ssmp extension is effectively disabled, and any attempt to access SPMP-related registers results in reads returning zero, and writes being ignored.



Figure 5. mpmmpdeleg CSR format.



The mpmmpdeleg.pmpnum field is a WARL field, which allows an implementation to hardwire the partition between PMP and SPMP.

Addressing:

Both PMP and SPMP entries are indexed starting from zero. For example, in an implementation with 64 total entries where pmpnum is configured to N ($N \leq 64$):

1. PMP entries 0 through $N-1$ act as PMP (i.e., PMP[0.. $N-1$]) and are accessible via standard PMP CSRs (e.g., pmpcfg[0..3] and pmpaddr[0..15] for RV32; pmpcfg[0,2] and pmpaddr[0..15] for RV64).
2. The remaining $64-N$ entries are delegated as SPMP (i.e., SPMP[0.. $63-N$]) and are indirectly accessed via xiselect (see [Section 2.7](#) and [Section 4.2](#)).
3. Accesses to out-of-range indices, such as reading PMP[N] or writing to SPMP[$64-N$] in this scenario,

are handled as follows: reads return zero, and writes are ignored.

Configuration registers:

The configuration register `spmpcfg[i]` of an SPMP entry is SXLEN-bit. Its lower 8 bits are an alias for the 8-bit field in the corresponding M-mode PMP configuration register.

Reconfiguration of delegated entries:

1. M-mode software can dynamically adjust the allocation between PMP and SPMP by writing to the `mpmpdeleg` CSR.
2. The `pmpnum` value cannot be set to an index that is less than or equal to that of any locked PMP entry. For instance, if `PMP[7]` is locked, any attempt to write a value less than 8 to `pmpnum` is ignored, and the field retains its prior value.
3. The `pmpnum` value can be set to override locked SPMP entries. For example, if `SPMP[0]` is locked, M-mode software can still increment `pmpnum`.
4. As `pmpnum` is reconfigured, SPMP entries with the largest indices change. For instance, raising `pmpnum` from `N` to `N+2` truncates SPMP and `spmpen` ranges from `SPMP[0..63-N]`, `spmpen[0..63-N]` to `SPMP[0..61-N]`, `spmpen[0..61-N]`. Conversely, reducing `pmpnum` from `N+2` to `N` expands SPMP and `spmpen` ranges from `SPMP[0..61-N]`, `spmpen[0..61-N]` to `SPMP[0..63-N]`, `spmpen[0..63-N]`.



Changing `pmpnum` at runtime carries the side effect of inheriting configuration values from previous PMP or SPMP roles. Consequently, before writing a new value to `pmpnum`, the M-mode software should perform confidence checks, including storing and clearing the fields of the affected configuration registers.

4.2. The Access Methods for SPMP CSRs in M-mode

M-mode employs different methods to access PMP and SPMP entries. PMP entries are accessed directly through their dedicated CSRs (i.e., `pmpcfg` and `pmpaddr`). Delegated SPMP entries, however, are accessed indirectly using the `xiselect` CSR (i.e., `siselect` and `miselect`).

For these indirect accesses, `miselect` selects the target SPMP entry, `mireg` accesses its `spmpaddr` register, and `mireg2` accesses its `spmpcfg` register. The `mireg3` through `mireg6` are reserved (i.e., reads return zero, writes are ignored).

The lock bit (`spmpcfg[i].L`) of an SPMP entry can only be cleared by M-mode through an indirect access using `miselect`.

The view provided by `miselect` is identical to that of `siselect` (see [Section 2.7](#)). For instance, if `N` ($N \leq 64$) out of 64 entries are delegated, `SPMP[0..N-1]` is accessed by writing `0x100+i` to the corresponding `xiselect` register (`siselect` for S-mode, `siselect` and `miselect` for M-mode), where `i` represents the entry index ($0 \leq i \leq N-1$). Any access attempt by either mode to an index outside this range (i.e., $i \geq N$) results in a read of zero, and writes are ignored.

<code>miselect</code> value	indirect CSR access of <code>mireg</code>
<code>0x100</code>	<code>mireg</code> → <code>spmpaddr[0]</code> , <code>mireg2</code> → <code>spmpcfg[0]</code>
<code>0x101</code>	<code>mireg</code> → <code>spmpaddr[1]</code> , <code>mireg2</code> → <code>spmpcfg[1]</code>
...	...

misselect value	indirect CSR access of mireg
0x13F	mireg → smpaddr[63], mireg2 → smpcfg[63]

Memory access in M-mode: If the effective privilege mode of the memory access is M, the access is allowed regardless of the SPMP permissions.

If the Shbare extension is implemented in conjunction with Ssmp, then when V=1 and hgap.MODE=Bare, the Hypervisor Virtual Machine Load and Store instructions (HLV, HLVX, HSV) executed from M-mode are subject to SPMP checks.

Chapter 5. Recommended Programming Guidelines

Two primary models guide the configuration of SPMP for isolating user-mode tasks from each other and from the S-mode operating system (OS). The selection of a model hinges on whether the number of available SPMP entries is sufficient to simultaneously contain all memory regions required by both user tasks and the OS.

- **Static Configuration:** This model involves programming all SPMP entries once at system initialization. It is predicated on the availability of enough entries to statically map all memory regions for both user tasks and the OS. This approach is only viable if the `Sspmpen` extension is implemented.
- **Dynamic Configuration:** This model requires reprogramming SPMP entries during each context switch. It is employed when the SPMP entry count is insufficient to concurrently map all task memory regions, necessitating dynamic updates to maintain memory isolation.

5.1. Static Configuration

The static configuration model is applicable when the number of SPMP entries is sufficient to map all memory regions for both user-mode tasks and the OS. Under this model, SPMP entries are configured a single time at system initialization and are not modified at runtime. Consequently, context switches between user-mode tasks only require updating the `spmpen` register(s).

During the boot process, machine-mode software is responsible for allocating SPMP entries and configuring them with the address ranges and permissions required by supervisor-mode software.

The OS then proceeds to populate the `spmpaddr[i]` and `spmpcfg[i]` CSRs with the specific address ranges and permissions for each user-mode task.

To protect its own memory, the OS also configures a set of SPMP entries for its address space, ensuring the `spmpcfg[i].U` (User) bit is cleared for these entries. Following initialization, the OS must activate its own entries by setting the corresponding bits in the `spmpen` register.

Before dispatching the first user task, the OS activates the SPMP entries assigned to that task by setting the appropriate bits in `spmpen`. During a context switch, the OS atomically deactivates the outgoing task's entries and activates the incoming task's entries using the `CSRRC` and `CSRRS` instructions, respectively. On RV32 systems, `spmpen` and `spmpenh` must be updated as an uninterruptible sequence to guarantee complete protection.

5.2. Dynamic Reconfiguration

The dynamic model is necessary when the number of available SPMP entries is insufficient to concurrently map all memory regions for the supervisor and all user tasks. Consequently, the OS must dynamically reconfigure the SPMP entries assigned to user tasks at every context switch. It is important to note that the number of SPMP entries on a hart must still be adequate to hold all supervisor entries plus the entries for one user-mode task at any given time.

The following sequence details the recommended procedure for dynamic reconfiguration:

1. **Disable Outgoing Task Entries.** Disable the SPMP entries for the outgoing task using a bitmask of its active entries, which is typically maintained in the task's control block. If the `Sspmpen` extension is available, the OS clears the relevant bits in `spmpen` via a `CSRRC` instruction. Otherwise,

the OS clears the `spmpcfg[i].A` field for each of the task's entries.

2. **Update SPMP Address Registers.** Program the `spmpaddr[i]` CSRs with the memory region boundaries of the incoming task.
3. **Update SPMP Configuration Registers.** For each relevant `spmpcfg[i]` field:
 - First, clear the existing configuration bits with a CSRRC instruction.
 - Then, set the new configuration bits with a CSRRS instruction.
4. **Enable Incoming Task Entries.** Activate the SPMP entries for the incoming task using its corresponding bitmask. If the `Ssmpen` extension is implemented, the OS sets the bits in `smpen` with a CSRRS instruction. Otherwise, the OS sets the `spmpcfg[i].A` field for each of the new task's entries.



SPMP entries configured to protect the supervisor, which are identified by `spmpcfg[i].U == 0`, should be treated as resident. It is highly recommended that these entries not be reprogrammed during the context switch procedure. Keeping supervisor entries persistent minimizes reconfiguration overhead and guarantees the consistent enforcement of supervisor memory protection.

5.3. Entry Configuration Recommendations

Programming SPMP entries involves a trade-off between the Naturally Aligned Power-of-Two (NAPOT) and Top-of-Range (TOR) address-matching modes (see [Section 2.3](#)).

While NAPOT provides a compact, single-entry encoding for power-of-two-aligned regions, it can cause internal fragmentation if the allocated region is larger than required, leading to memory inefficiency. Conversely, TOR mode typically requires two entries to define an arbitrary region (base and top), which can consume the available entries more rapidly. However, TOR may prove more entry-efficient for certain power-of-two aligned regions.

This trade-off is particularly pronounced in Microcontroller Unit (MCU) systems, where memories often have a sparse, fixed mapping. In such scenarios, a rigid adherence to either NAPOT or a naive pairing of TOR entries can be inefficient or create unintended protection gaps. Furthermore, using consecutive or overlapping TOR entries to define multiple regions with distinct permissions can introduce subtle and hazardous dependencies. For example, sharing a `spmpaddr` register as a boundary between a supervisor and a user region means that shrinking the supervisor region could unintentionally enlarge the user region. Likewise, modifying a shared boundary during a context switch might inadvertently expose protected memory.

To mitigate these risks, the following disciplined configuration model is recommended:

- Exclusively use the TOR mode, and treat each even/odd indexed pair of `spmpaddr[i]` registers as a single base/top definition for a memory region.
- Establish a convention where SPMP entries are managed in pairs, such as (0, 1), (2, 3), ..., (62, 63). Region activation should only be controlled via the odd-indexed entries (e.g., `smpen[1]`, `smpen[3]`), as each odd entry completes a region's definition.
- Allocate SPMP entry pairs in descending order of their indices, from highest to lowest. This strategy, which corresponds to an ascending order of priority, permits the OS to create temporary, higher-priority subregions using lower-indexed entries without disturbing existing configurations.

Adhering to this structured approach with TOR-mode entries fosters clearer isolation boundaries,

minimizes the risk of configuration errors, and enhances the runtime flexibility of the memory protection scheme.

5.4. Reconfiguration Non-preemption and Synchronization

To maintain the integrity of the SPMP configuration, the entire reconfiguration sequence during a context switch must execute atomically as a non-preemptible critical section. This is necessary because the process involves modifications to multiple CSRs, and any interruption could leave the system in an inconsistent or insecure state.

For the **dynamic reconfiguration model**, this critical section must encompass all updates to `smpaddr[i]`, `smpcfg[i]`, and `smpen`. For the **static configuration model**, atomicity is a concern only on RV32 systems with over 32 SPMP entries, which require a coordinated update of both `smpen` and `smpenh`.

To block asynchronous interrupts during this process, the supervisor must temporarily clear the SIE (Supervisor Interrupt Enable) bit in the `sstatus` CSR.

By enforcing non-preemption and correct synchronization, the software guarantees that SPMP enforcement remains deterministic, secure, and verifiable across all context switches.